

Конспект для подготовки к собеседованию

Data Science (на основе видео и ML Handbook Яндекса)

Вот расширенный и глубоко проработанный конспект для подготовки к собеседованию. Я значительно детализировал цитаты из видео, добавил экспертные «фишки», о которых говорит спикер, и обогатил теоретическую часть математическими выкладками и строгими концепциями из ML Handbook.

1. Как работает логистическая регрессия.

Цитата из видео: «У нас транспонированный вектор весов признаков умножается на наш объект. Мы пропускаем это всё через сигмоиду и получаем вероятность принадлежности классу. <...> Фишечка: расскажите почему именно логит. То, что это приходит из теории вероятности, из максимизации правдоподобия. Расскажите то, что он выпукл и вы будете всегда сходиться к одному решению. <...> На больших ошибках предсказаний он даёт большой штраф».

Дополнение из учебника: Логистическая регрессия предсказывает логит (log odds) — логарифм отношения вероятностей:

$$\log \left(\frac{p}{1-p} \right)$$

Чтобы перевести линейный ответ $\langle w, x_i \rangle$ в вероятность, используется сигмоида:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Оптимизация LogLoss — это следствие метода максимального правдоподобия для распределения Бернулли:

$$p(y|X, w) = \prod p_i^{y_i} (1 - p_i)^{1-y_i}$$

Разделяющая поверхность классификатора задается уравнением $\langle w, x \rangle = 0$, что геометрически является гиперплоскостью.

Резюме: Это линейный классификатор, обучаемый через максимизацию правдоподобия (минимизацию строго выпуклого LogLoss). Модель предсказывает логарифм отношения шансов, который через сигмоиду трансформируется в вероятность, а граница классов образует гиперплоскость.

2. Что такое переобучение и как его замечать.

Цитата из видео: «Когда наша модель слишком сильно подстроилась под данные: у нас есть неплохое качество на трейне, но на тесте качество слишком плохое. <...> Более продвинутый ответ — это использовать кросс-валидацию, посмотреть, что модель подстроилась под различные фолды. <...> У каждой модели есть своя специфика борьбы: у деревьев это высота листа, у нейронных сетей — дропаут».

Дополнение из учебника: В терминах теории ML переобучение описывается через Bias-Variance Decomposition. Переобученная модель имеет высокий разброс (Variance) — она избыточно чувствительна к случайным флуктуациям и запоминает шум обучающей выборки вместо реального сигнала.

Резюме: Переобучение (Overfitting) — это состояние модели с высоким Variance, когда она заучивает шум. Детектируется через разрыв метрик между train и test/фолдами кросс-валидации. Лечится регуляризацией, зависящей от типа алгоритма.

3. Фишки: data augmentation, SMOTE, шум.

Цитата из видео: «Круто работать с переобучением через данные: можно добавить шум, можно сделать больше новых данных... использовать SMOTE, чтобы разнообразить ваш датасет. Это всё очень хорошие методы борьбы с переобучением».

Дополнение из учебника: Аугментация (внесение изменений в данные) — важнейший инструмент регуляризации в глубинном обучении. Например, поворот картинки самолета на 10 градусов создает новый для сети объект, но с той же меткой. Это позволяет «дать модели понять, какие преобразования над данными являются допустимыми». Главное ограничение: аугментированные данные должны принадлежать к той же генеральной совокупности (например, сильное инвертирование цветов может навредить, если в реальности таких данных нет).

Резюме: Аугментация и искусственное зашумление данных — это мощная форма регуляризации, расширяющая выборку и обучающая модель инвариантности к допустимым в реальном мире искажениям (повороты, шумы).

4. Почему L1 регуляризация зануляет признаки.

Цитата из видео: «Интуитивное понимание: гиперплоскость L1 регуляризации — это октаэдр, а L2 — сфера. И октаэдр сводится к нулям. <...> Более крутое доказательство — это через субдифференциалы: мы доопределяем производную по модулю. У L2 идут большие штрафы при больших весах, а для модуля градиент более равномерный. Поэтому веса равномерно отнимаются и сходятся к нулю».

Дополнение из учебника: При L1-регуляризации (Lasso) штраф имеет вид:

$$\lambda \sum |w_j|$$

Линии уровня L1-нормы — это ромбы (в 2D) или многогранники с острыми вершинами, лежащими строго на осях координат. Линии уровня основной функции потерь в точке оптимума касаются именно этих углов на осях, обнуляя часть координат. Оптимизация L1 сложна из-за недифференцируемости модуля в нуле, для чего применяются проксимальные методы градиентного спуска.

Резюме: Геометрия L1-штрафа (ромбы с острыми углами на осях) и свойства субградиента приводят к тому, что точка касания с функцией потерь попадает на оси координат, создавая разреженные (зануленные) вектора весов, что работает как встроенный feature selection.

5. Как устроено дерево решений.

Цитата из видео: «Это бинарное дерево. Для категорий есть ответы да/нет. Для вещественных признаков мы разбиваем их на бины, проходим по порядку и смотрим: если признак меньше значения — идет влево, больше — вправо. Мы хотим уменьшать кроссэнтропию или увеличивать Information Gain (чтобы в листьях была как можно меньше дисперсия)».

Дополнение из учебника: Дерево задает кусочно-постоянную аппроксимацию. Внутренняя вершина содержит предикат:

$$B_v(x, j, t) = [x_j \leq t]$$

Жадный алгоритм на каждом шаге максимизирует критерий ветвления:

$$Branch = |X_m|H(X_m) - |X_l|H(X_l) - |X_r|H(X_r)$$

Информативность (impurity) $H(X)$ зависит от задачи:

- для регрессии (MSE) это дисперсия ответов: $\frac{1}{|X|} \sum (y_i - \bar{y})^2$
- для классификации — критерий Джини: $\sum p_k(1 - p_k)$
- или энтропия Шеннона: $-\sum p_k \log p_k$

Резюме: Дерево строится жадно: пространство признаков итеративно делится гиперплоскостями, параллельными осям. Сплит выбирается так, чтобы максимизировать падение неопределенности (дисперсии таргета или энтропии классов) в дочерних узлах.

6. Переобучение в деревьях, прунинг.

Цитата из видео: «Дерево можно ограничить не с помощью кросс-валидации, а при построении: добавить в лосс наказание плюс α умножить на высоту дерева. Расскажите про прунинг. Можно требовать минимальное количество элементов в листе при разбитии».

Дополнение из учебника: Деревья — чемпионы переобучения, они могут идеально приблизить обучающую выборку (до 1 объекта в листе), полностью потеряв обобщающую способность. Регуляризация (early stopping/пре-прунинг) останавливает ветвление при достижении лимитов глубины или размера листа. Прунинг (стрижка) работает иначе: дерево строится жадно до конца, а затем вершины удаляются с проверкой качества на отложенной выборке.

Резюме: Деревья легко запоминают шум. С этим борются либо останавливая построение (early stopping по гиперпараметрам глубины и листьев), либо отсекая лишние ветви уже построенного глубокого дерева на отложенной выборке (прунинг).

7. Как работает случайный лес (Random Forest).

Цитата из видео: «Деревья строятся параллельно и независимо друг от друга. Используется бутстрап объектов. Важная фишка: признаки выбираются бутстрапом *на каждом разбиении* (в каждом узле), а не для всего дерева в начале. Лес нужен, чтобы уменьшать дисперсию ошибки (variance), поэтому нам нужны как можно более глубокие, переобученные деревья с минимальным смещением».

Дополнение из учебника: В основе лежит метод случайных подпространств и бэггинг. В каждой вершине отбираются случайные $n < N$ признаков. Если базовые глубокие деревья не скоррелированы, то дисперсия ансамбля из k деревьев падает в k раз:

$$\mathbb{V}[a(x, X)] = \frac{1}{k} \mathbb{V}_X b(x, X)$$

Смещение (bias) леса при этом остается равным смещению одного дерева.

Резюме: Случайный лес — параллельный ансамбль глубоких переобученных деревьев (с низким bias). Рандомизация по объектам (бутстрап) и признакам (в каждом узле) делает деревья независимыми, что при усреднении их ответов радикально снижает общую дисперсию (variance) модели.

8. Чем отличается бустинг от Random Forest.

Цитата из видео: «Бустинг строится последовательно, каждая модель учится на ошибку предыдущей. Если выкинуть первую модель, бустинг сломается, а случайный лес — нет. Бустинг уменьшает смещение (bias), поэтому мы хотим, чтобы дисперсия каждой базовой модели была как можно меньше — используются неглубокие деревья (от 3 до 8 уровней)».

Дополнение из учебника: Градиентный бустинг строит композицию аддитивно. Каждое следующее дерево настраивается на минимизацию антиградиента функции потерь композиции всех прошлых шагов. В отличие от леса, где деревья учатся независимо, бустинг требует простых базовых алгоритмов (weak learners) для постепенного снижения Bias, но подвержен риску переобучения при слишком большом числе итераций.

Резюме: Лес снижает Variance, усредняя параллельные глубокие деревья. Бустинг снижает Bias, строя последовательность мелких деревьев, каждое из которых корректирует остаточную ошибку предыдущих.

9. Метрики регрессии и классификации.

Цитата из видео: «Регрессия: MAE, MAPE, MSE, RMSE, SMAPE. Важно говорить, что они значат (MSE подстраивается под выбросы, MAPE нужна для понимания порядка). Классификация: Accuracy, Precision, Recall, F1, F-beta, ROC-AUC, PR-кривая».

Дополнение из учебника: • MSE математически приближает матожидание целевой переменной.

- MAE приближает медиану и значительно меньше штрафует за грубые выбросы.
- MAPE применяется, когда важна процентная ошибка (прогноз цен, кассовых сборов).
- Accuracy неинформативна при дисбалансе классов.

Резюме: Для регрессии метрики выбираются исходя из отношения к выбросам (MAE/MSE) и необходимости относительных оценок (MAPE). В классификации базой является матрица ошибок; при дисбалансе фокус смещается с Accuracy на метрики полноты, точности и ранжирования.

10. Что такое ROC-AUC и как его интерпретировать.

Цитата из видео: «ROC-AUC показывает: если мы берём случайный объект из класса 1 и случайный объект из класса 0, наша модель их правильно отранжировала. Вопрос-ловушка: как изменится ROC-AUC, если предсказания 0.8 и 0.3 заменить на 0.9 и 0.7? Ответ: никак, нам не важна сама вероятность, важно лишь расположение (ранжирование) положительного класса относительно отрицательного».

Дополнение из учебника: Это площадь под кривой (Area Under Curve), отражающей зависимость True Positive Rate от False Positive Rate при всевозможных порогах классификатора. Дает агрегированную оценку качества именно ранжирующей способности модели.

Резюме: ROC-AUC — это вероятность правильного попарного ранжирования: случайно взятый положительный объект получит скор выше, чем случайный отрицательный. Метрика нечувствительна к абсолютному значению вероятностей (калибровке).

11. Precision — как объяснить на собеседовании.

Цитата из видео: «Точность. Нарисуйте для себя Confusion Matrix, чтобы не путать

False Positive и False Negative. Пример: выбор места для постройки дома. Нам не страшно пропустить хорошие места, но очень важно, чтобы то место, которое мы определили как "хорошее" (положительный вердикт модели), гарантированно не было подвержено катаклизмам».

Дополнение из учебника: Доля истинно положительных (True Positives) среди всех объектов, названных классификатором положительными. Служит для контроля ошибок первого рода (ложных срабатываний).

Резюме: Точность измеряет "надежность" положительных срабатываний модели. Метрика критична в задачах, где цена ложной тревоги (False Positive) катастрофически высока.

12. Recall и его применение в медицине.

Цитата из видео: «Пример: в медицине нам очень важно не пропустить больного, всех больных мы хотим найти. Для нас не так страшно, если мы здорового отправим на дополнительное обследование (False Positive), главное — человек выживет».

Дополнение из учебника: Полнота (Recall) характеризует способность модели находить целевые объекты из всего множества реально существующих положительных примеров. Фокусируется на минимизации ошибок второго рода (False Negatives).

Резюме: Полнота измеряет "охват" класса. Ее максимизируют, когда пропуск целевого события (False Negative) обходится намного дороже, чем ложное срабатывание.

13. Связь Recall и ROC-AUC.

Цитата из видео: «Фишечка: Recall на самом деле используется в ROC-AUC. Осями там выступают True Positive Rate (это в чистом виде Recall для класса 1) и False Positive Rate (это по сути "Recall для класса 0")».

Дополнение из учебника: Кривая строится в координатах (FPR, TPR) при изменении порога уверенности. TPR (ось Y) математически тождественна метрике Recall.

Резюме: ROC-кривая буквально состоит из метрик полноты: она показывает, как меняется полнота нахождения целевого класса (TPR) ценой потери полноты правильного нахождения отрицательного класса (рост FPR).

14. Зачем нужна PR-кривая и в чём её отличие от ROC-AUC.

Цитата из видео: «При суперсильном дисбалансе классов PR-кривая показывает себя лучше. ROC-AUC более оптимистичен: он говорит про предсказание как положительных, так и отрицательных примеров, а отрицательных бывает слишком много. PR-кривая делает акцент именно на положительных таргетах».

Дополнение из учебника: PR-кривая (Precision-Recall) концентрируется исключительно на качестве предсказания меньшего (положительного) класса, игнорируя успехи модели в нахождении True Negatives, которых в задачах с перекосом подавляющее большинство.

Резюме: В условиях жесткого дисбаланса ROC-AUC дает обманчиво высокие оценки (огромное число легких нулей сдерживает рост FPR). PR-кривая фокусируется только на миноритарном классе, давая объективную картину.

15. Подбор гиперпараметров (GridSearch, RandomSearch, Optuna).

Цитата из видео: «GridSearch не подходит, если большой набор значений. RandomSearch берет случайные значения. Байесовская оптимизация (Optuna, Hyperopt) строит распределение потенциальной награды... она балансирует exploration и exploitation (смотрит, где потенциальный эффект, и где модель уже показала себя хорошо)».

Дополнение из учебника: GridSearch страдает от размерности. RandomSearch эффективнее находит оптимум (60 итераций дают вероятность 95% попасть в топ-5% лучших решений). Байесовская оптимизация строит суррогатную вероятностную модель и функцию выгоды (Expected Improvement):

$$EI(x) = \frac{l(x)}{g(x)}$$

Еще есть PBT (Population Based Training) из эволюционных стратегий.

Резюме: Перебор по сетке слишком долгий. Случайный поиск работает лучше за счет исследования пространства. Индустриальный стандарт — Байесовская оптимизация (алгоритм TPE), которая учится на истории запусков.

16. Как оценить важность признаков (feature importance) в модели.

Цитата из видео: «В бизнесе важно показывать, почему ML работает. Линейные: коэффициенты (при масштабировании). Деревья: индекс Джини (как часто и много разбиения по признаку давали gain). Permutation Importance: перемешали значения фичи, посмотрели, насколько упало качество. SHAP values (основан на теории игр) — строит подвыборки, делает дисперсионный анализ. Минус: асимптотика, работает очень долго, но лучше всех показывает правду».

Дополнение из учебника: В линейной модели веса w_i имеют физический смысл и их величина интерпретируется как важность, но только при отсутствии мультиколлинеарности.

Резюме: Базовые методы — веса в линейных моделях или InfoGain в деревьях. Универсальные методы — Permutation и SHAP. SHAP математически самый точный, но вычислительно тяжелый.

17. Как работать с категориальными фичами (и чем хорош CatBoost).

Цитата из видео: «Базовые: Label Encoding, One-Hot (когда мало уникалов), Frequency, Target Encoding (среднее таргета для категории). Экспертный ответ про CatBoost: 1. Он сам берет пары (комбинации) категориальных фичей. 2. Ordered boosting: чтобы не было переобучения (target leakage), статистики для объекта считаются только по тем объектам, которые модель "видела" раньше. Это дает эффект active learning».

Дополнение из учебника: При классическом One-Hot Encoding возникает риск мультиколлинеарности — сумма созданных столбцов равна единице, что создает линейную зависимость и ломает регрессию (dummy variable trap).

Резюме: Простые методы кодируют категории числами, ONE — векторами. Продвинутое модели (CatBoost) генерируют комбинации признаков и используют динамическое таргет-кодирование.

18. В чём разница между == и is в Python.

Цитата из видео: «Базовый ответ: == смотрит на равенство значений объектов, а is проверяет, один это объект или нет, равны ли адреса памяти».

Дополнение из учебника: Различие между операторами сравнения значений (`==`) и идентификаторов в памяти (`is`) — это ключевой инженерный нюанс Python.

Резюме: `==` проверяет данные (содержимое), а `is` проверяет идентичность ссылок.

19. Как устроен dict в Python и что такое коллизии.

Цитата из видео: «Dict — это хэш-мапа. Асимптотика в среднем $O(1)$. С помощью хэш-функции берем остаток от деления на размер дикта и получаем индекс. Коллизия — когда для разных ключей получается один индекс. В Java используются списки. В Python используются открытая адресация: складываем в следующие индексы. При заполнении на $2/3$ дикт увеличивается в 2 раза».

Дополнение из учебника: Использование хэш-таблиц обеспечивает быстрый поиск. Понимание механизмов коллизий необходимо для написания эффективного кода.

Резюме: Словарь — хэш-таблица на открытой адресации. При совпадении хэшей Python ищет ближайшую свободную ячейку. Массив расширяется в 2 раза при заполнении на 66%.

20. Что такое итераторы и зачем они нужны.

Цитата из видео: «Итератор — объект, который позволяет обойти все элементы коллекции, не держа всю её в памяти. Обязательны магические методы `__iter__` и `__next__`. Нужны для оптимизации использования оперативной памяти».

Дополнение из учебника: Итераторы и генераторы реализуют концепцию "ленивых" вычислений (lazy evaluation). Это критически важно в машинном обучении для экономии RAM при потоковой загрузке датасетов.

Резюме: Итераторы вычисляют элементы по одному "на лету". Это экономит оперативную память, позволяя обрабатывать Big Data.

21. Теория вероятностей: задача на выпадение суммы 11.

Цитата из видео: Вопрос-задача для комментариев: «Какая вероятность того, что при подбрасывании двух шестигранных кубиков выпадет сумма 11?».

Дополнение из учебника: Подбрасывания кубиков — статистически независимые события. Общее количество исходов: $6 \times 6 = 36$. Чтобы получить сумму 11, есть только два благоприятных исхода: (5, 6) и (6, 5).

Резюме: Вероятность равна количеству благоприятных исходов, деленному на общее количество исходов:

$$P = \frac{2}{36} = \frac{1}{18}$$

22. Теорема Байеса: формула и применение.

Цитата из видео: «Почти всегда выпадает задача на теорему Байеса. Сакральных знаний не нужно, просто заучите формулу и определитесь с событиями».

Дополнение из учебника: Формула:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Формула корректирует априорные ожидания с учетом новых фактов. Пример с болезнью показывает, почему точный тест на редкую болезнь часто врет.

Резюме: Переоценивает вероятности гипотез при поступлении новых данных. Особенно важна для понимания парадоксов вероятности.

23. Что такое p-value и как его объяснить доступно.

Цитата из видео: «p-value — это вероятность получить наблюдаемое различие между выборками или более экстремальное в случае, если нулевая гипотеза верна».

Дополнение из учебника: Это фундаментальный показатель в проверке статистических гипотез (A/B тестах). Оценивает противоречие данных нулевой гипотезе.

Резюме: Это метрика "удивления". Чем меньше p-value, тем маловероятнее получить результаты случайно, что дает основания отвергнуть нулевую гипотезу.

24. Какие статтесты бывают и когда их использовать.

Цитата из видео: «t-test Стьюдента — когда нормальное распределение. Критерий Манна-Уитни — когда не нормальное и есть выбросы. Bootstrap — работает во всех случаях. Экспертно: сказать про стратификацию и CUPED».

Дополнение из учебника: Выбор теста требует понимания ограничений по типу распределения и объему выборки.

Резюме: Выбор теста зависит от распределения: параметрические (t-test) для нормальных средних, непараметрические (Манна-Уитни) для медиан. CUPED используется для повышения чувствительности.

25. Функции активации в нейросетях и работа Dropout при инференсе.

Цитата из видео: «Активации (tanh, sigmoid, relu) добавляют нелинейность, влияют на градиенты. Dropout борется с переобучением. При инференсе работают все нейроны, изменения делаем во время трейна».

Дополнение из учебника: Без активаций слои сводятся к линейной регрессии. Инициализация весов (Xavier, Kaiming) зависит от функции активации. Dropout зануляет нейроны на трейне. На инференсе выходы масштабируются на $(1 - p)$:

$$x_{inf} = x_{train} \cdot (1 - p)$$

или используется Inverted Dropout.

Резюме: Функции активации ломают линейность слоев. Dropout обучает ансамбль подсетей. На инференсе сеть работает целиком с масштабированием сигналов.